Projet Tln 324 - BE

Institut Polytechnique des Sciences Avancées (IPSA)

Projet d'introduction à l'intelligence artificielle

Réalisé par :

Florian Alaux, Guilhem Perret-bardou

Encadrant :

M. S.LABSIR

Année académique : 2024-2025

10 mars 2025

Table des matières

Introduction **3**

1 Détection de SPAM **3**
1.1 Pre-Processing . . . 3
1.2 Implémentation des algorithmes . . . 4
1.2.1 KNN . . . 4
1.2.2 Naive Bayes . . . 4
1.2.3 LDA . . . 6
1.2.4 Pour aller plus loin ! . . . 8

2 Détection de cancer du sein **9**
2.1 Pre-Processing . . . 9
2.2 Un peu de théorie sur la régression logistique . . . 10
2.3 implémentation RL . . . 12

Conclusion **16**

Listings

1 Chargement du jeu de données SPAM 3
2 Implementation de KNN 4
3 Implementation de Naive Bayes 6
4 Implémentation de LDA 7
5 Chargement du jeu de données Breast Cancer 10
6 Implementation de la regression logistique personnalisée 13

Introduction

Ce bureau d'étude a pour objectif d'étudier les différentes méthodes abordées dans le cours : l'algorithme des k plus proches voisins (KNN) et la règle de classification bayésienne.
L'objectif de ce projet est d'implémenter, tester et comparer les algorithmes d'apprentissage automatique étudiés en cours sur différents jeux de données. Le projet sera divisé en deux parties :

- **KNN et classificateur statistique sur le jeu de données SPAM :** dans la première partie, les méthodes des k plus proches voisins (KNN), Naïve Bayes et LDA seront développées et testées sur un jeu de données de courriers indésirables (SPAM).
- **Comparaison avec la régression logistique :** dans la seconde partie, des données plus complexes seront utilisées, et les algorithmes précédemment développés seront comparés à une méthode pionnière des réseaux de neurones : la régression logistique.

1 Détection de SPAM

Dans la première partie, nous visons à traiter le problème de la détection de la présence de spam dans les e-mails reçus. Pour résoudre ce problème, nous allons développer et tester des algorithmes de classification spécialement conçus pour classer un message en spam ou en ham.
Pour cela, nous proposons d'utiliser un jeu de données fourni par l'Université d'Irvine à l'adresse suivante : `https://archive.ics.uci.edu/dataset/228/sms+spam+collection`.

1.1 Pre-Processing

Dans l'en-tête, un extrait de code permettant de charger le jeu de données est fourni. Le jeu de données contient deux types de données : les messages classés comme SPAM (classe 1) et les messages de confiance, également appelés HAM (classe 0).

Listing 1: Chargement du jeu de données SPAM

```python
#---------------------------------PROJET : PARTIE I--------------------#

import sklearn
import numpy as np
import pandas as pd
from sklearn.feature_extraction.text import CountVectorizer
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report, euclidean_distances
import zipfile
import requests
from io import BytesIO


# Telecharger le fichier ZIP contenant le jeu de donnees
url = "https://archive.ics.uci.edu/ml/machine-learning-databases/00228/smsspamcollection.zip"
response = requests.get(url)  # Recuperation du fichier depuis l'URL
zip_file = zipfile.ZipFile(BytesIO(response.content))  # Chargement du fichier ZIP

# Extraction du fichier SMSSpamCollection
with zip_file.open('SMSSpamCollection') as file:  # Ouverture du fichier dans le ZIP
    df = pd.read_csv(file, sep='\t', names=['label', 'message'])

# Conversion des labels : 'ham' devient 0, 'spam' devient 1
df['label'] = df['label'].map({'ham': 0, 'spam': 1})

# Sauvegarde du jeu de donnees pretraite
df.to_csv('SMSSpamCollection_processed.csv', index=False)

# Definition des variables
X = df['message']
y = df['label']
```

Pour traiter les données SPAM, il est nécessaire d'extraire les informations de chaque message et de les convertir en un format numérique. Par exemple, si les données sont constituées de deux phrases « It is false », « Is is true », le vocabulaire est {« it », « is », « false », « true »}. Nous voulons transformer chaque phrase en une ligne contenant le nombre de fois que chaque mot du vocabulaire apparaît.

$$\text{« It is false »} \Rightarrow [1, 1, 1, 0]$$

$$\text{« It is true »} \Rightarrow [1, 1, 0, 1]$$

à partir de ce principe la on peut créer un dictionnaire de mot et donc Vectoriser les messages pour transformer les données de SPAM en un ensemble de valeurs de la forme $\{\mathbf{x}_i\}_{i=1}^N$ tel que $x_i \in \mathbb{R}^{8713}$

1.2 Implémentation des algorithmes

Maintenant, nous proposons de classifier les données traitées en utilisant divers algorithmes de classification. Étant donné l'ensemble d'apprentissage $\{\mathbf{x}_i\}_{i=1}^N$ avec des classes connues $\{y_i\}_{i=1}^N \in \{\underset{\text{spam}}{0}, \underset{\text{ham}}{1}\}^N$, nous visons à prédire la classe des données de test $\mathbf{x}^*$.

1.2.1 KNN

Pour commencer, nous voulons implémenter l'algorithme des K-Nearest Neighours (KNN), qui trouve le voisin d'apprentissage le plus proche des données de test selon une métrique de distance spécifiée. Pour ce faire, une classe appelée `KNN` a été pré-créée et contient trois méthodes.

Listing 2: Implementation de KNN

```python
class KNN:
    def __init__(self, k=3):
        self.k = k

    def fit(self, X, y):
        self.X_train = X
        self.y_train = y

    def predict(self, X):
        y_pred = [self._predict(x) for x in X]
        return np.array(y_pred)

    def _predict(self, x):
        distances = euclidean_distances(self.X_train, [x]).flatten()    #Calcul de distance
        indices_proches = np.argsort(distances)[:self.k]    #classe en ordre croissant et garde des k
            premiers (donc les plus proches)
        labels_proches = self.y_train[indices_proches]    #on garde les labels des k voisins
        return np.bincount(labels_proches).argmax()     #compte chaque classe et ressort celle qui
            apparait le plus
```

En utilisant la fonction "Accuracy-Score" On peut obtenir les résultats suivants :

Taille du jeu d'entraînement	Accuracy
557	0.8686
1671	0.8893
2786	0.9024
4457	0.9256

Les résultats obtenu correspondent à nos attentes, en effet plus la taille du jeu d'entraînement est grande, plus la précision de notre système est grande. En revanche on remarque que la variation de précision est uniquement de 5.7% alors que le temps d'exécution du programme quant à lui est beaucoup plus important pour obtenir une précision de 92.56% que pour une précision de 86.86%

1.2.2 Naive Bayes

Maintenant, nous proposons d'implémenter une méthode statistique appelée Naïve Bayes. Cette méthode suppose le modèle suivant pour la distribution a posteriori des données d'apprentissage $\mathbf{x}_i = \left[x_i^1, \ldots, x_i^P\right]^\top$

$$\mathbb{P}(y_i = k|\mathbf{x}_i) = \left(\prod_{p=1}^{P} p(x_i^p|y_i = k)\right) \underbrace{\mathbb{P}(y_i = k)}_{\pi_k} \quad \forall k \in \{0, 1\}$$

L'équation ci-dessus est obtenue grâce à **l'hypothèse d'indépendance conditionnelle des caractéristiques** (Naïve Bayes).
Cela signifie que, pour une classe $y_i = k$ donnée, les caractéristiques $x_i^1, x_i^2, \ldots, x_i^P$ sont supposées indépendantes. Cette hypothèse simplifie le calcul de la probabilité conditionnelle $\mathbb{P}(x_i|y_i = k)$ en un produit de probabilités marginales $\prod_{p=1}^{P} \mathbb{P}(x_i^p|y_i = k)$. Combinée à la probabilité a priori $\pi_k = \mathbb{P}(y_i = k)$, on obtient la formule de Bayes simplifiée ci-dessus
Dans ce qui suit, nous supposons que $p(x_i \mid y_i = k)$ suit une **distribution gaussienne** de moyenne μ_k^p et de variance $(\sigma_k^p)^2$. Cela nous permet d'obtenir l'équation ci-dessus.

En supposant l'indépendance de chaque $\mathbf{x}_i$.

Estimateur de μ_k^p :

On maximise la log-vraisemblance par rapport à $\widehat{\mu}_k^p$:

$$\frac{\partial \log \mathcal{L}}{\partial \mu_k^p} = \sum_{i:y_i=k} \frac{x_i^p - \mu_k^p}{(\sigma_k^p)^2} = 0$$

En résolvant :

$$\sum_{i:y_i=k} (x_i^p - \mu_k^p) = 0 \quad \Rightarrow \quad \boxed{\widehat{\mu}_k^p = \frac{1}{N_k} \sum_{i:y_i=k} x_i^p}$$

Estimateur de $(\sigma_k^p)^2$:

On dérive par rapport à $(\sigma_k^p)^2$:

$$\frac{\partial \log \mathcal{L}}{\partial (\sigma_k^p)^2} = \sum_{i:y_i=k} \left[-\frac{1}{2(\sigma_k^p)^2} + \frac{(x_i^p - \mu_k^p)^2}{2(\sigma_k^p)^4} \right] = 0$$

Simplification :

$$\sum_{i:y_i=k} \left[-1 + \frac{(x_i^p - \mu_k^p)^2}{(\sigma_k^p)^2} \right] = 0 \quad \Rightarrow \quad \boxed{(\widehat{\sigma}_k^p)^2 = \frac{1}{N_k} \sum_{i:y_i=k} (x_i^p - \widehat{\mu}_k^p)^2}$$

Estimateur de $\widehat{\pi}_k$:

La contrainte $\sum_k \widehat{\pi}_k = 1$ impose d'utiliser un multiplicateur de Lagrange. En maximisant :

$$\log \mathcal{L} = N_k \log \pi_k + \text{termes constants}, \quad \text{sous } \widehat{\pi}_k \geq 0.$$

Solution directe (proportion empirique) :

$$\boxed{\widehat{\pi}_k = \frac{N_k}{N}}$$

On peut montrer que les estimateurs sont donnés par :

$$\hat{\mu}_k^p = \frac{1}{N_k} \sum_{i:y_i=k} x_i^p, \quad (\hat{\sigma}_k^p)^2 = \frac{1}{N_k} \sum_{i:y_i=k} (x_i^p - \hat{\mu}_k^p)^2, \quad \hat{\pi}_k = \frac{N_k}{N}$$

où $N_k = \text{card}(\{\mathbf{x}_i \mid y_i = k\})$.
Une fois les estimateurs calculés, ils sont utilisés pour prédire la classe y^* de $\mathbf{x}^* = [x^{*1}, \ldots, x^{*P}]$ en maximisant la distribution a posteriori :

$$p(\mathbf{x}^* \mid y = k) = \left(\prod_{p=1}^{P} p(x^{*p} \mid y = k) \right) \hat{\pi}_k \quad \forall k \in \{0, 1\}.$$

On cherche à démontrer la règle de décision du classifieur Naive Bayes Gaussien :

$$y^* = \underset{k \in \{0,1\}}{\text{argmax}} \left(-0.5 \sum_{p=1}^{P} \frac{(x^{*p} - \widehat{\mu}_k^p)^2}{(\widehat{\sigma}_k^p)^2} + \log \widehat{\pi}_k \right)$$

Pour chaque classe k, la vraisemblance s'écrit :

$$p(\mathbf{x}^* | y = k) = \prod_{p=1}^{P} \frac{1}{\sqrt{2\pi (\widehat{\sigma}_k^p)^2}} \exp\left(-\frac{(x^{*p} - \widehat{\mu}_k^p)^2}{2(\widehat{\sigma}_k^p)^2} \right)$$

En prenant le logarithme (et en omettant les termes constants) :

$$\log\left(p(\mathbf{x}^* | y = k) \widehat{\pi}_k\right) = -0.5 \sum_{p=1}^{P} \frac{(x^{*p} - \widehat{\mu}_k^p)^2}{(\widehat{\sigma}_k^p)^2} + \log \widehat{\pi}_k$$

La classe prédite maximise cette expression :

$$\boxed{y^* = \underset{k \in \{0,1\}}{\text{argmax}} \left(-0.5 \sum_{p=1}^{P} \frac{(x^{*p} - \widehat{\mu}_k^p)^2}{(\widehat{\sigma}_k^p)^2} + \log \widehat{\pi}_k \right)}$$

Listing 3: Implementation de Naive Bayes

```python
class NaiveBayes:
    def fit(self, X, y):
        n_samples, n_features = X.shape
        self._classes = np.unique(y)
        n_classes = len(self._classes)
        self._priors = np.zeros(n_classes)
        self._means = np.zeros((n_classes, n_features))
        self._variances = np.zeros((n_classes, n_features))

        for idx, c in enumerate(self._classes):
            X_c = X[y == c]   # On garde uniquement les lignes de X appartenant a la classe c
            N_k = X_c.shape[0]
            # Les trois estimateurs de la question 8 :
            self._priors[idx] = N_k / n_samples  # \(\pi_k\)
            self._means[idx] = np.mean(X_c, axis=0)  # \(\mu_k^p\)
            self._variances[idx] = np.var(X_c, axis=0) + 1e-6  # \(\sigma_k^p^2 +\) petite valeur pour eviter les problemes numeriques
                                                                # 1e-6 pris car il apparait comme parametre pour LDA, peut etre modifie pour un autre nombre proche de 0
    def predict(self, X):
        y_pred = [self._predict(x) for x in X]
        return np.array(y_pred)

    def _predict(self, x):
        posteriors = []

        for idx, c in enumerate(self._classes):  # Separe en deux termes \( y^* \) car on nous donne "posteriors = []"
            prior = np.log(self._priors[idx])  # \(\log(\pi_k)\)
            lvg = -0.5 * np.sum((x - self._means[idx]) ** 2 / self._variances[idx])  # Log-vraisemblance gaussienne
            posteriors.append(prior + lvg)  # Additionne les deux log pour obtenir le score de la classe c

        return self._classes[np.argmax(posteriors)]  # Retourne la classe avec le plus grand score
```

Modèle	Accuracy
KNN	0.9256
Naive Bayes	0.9650

On remarque que l'accuracy de Naïve Bayes est bien plus performante que KNN (+ 4%.).Cet algorithme est également bien plus rapide donc plus intéressant que le précédent.

1.2.3 LDA

Nous proposons maintenant d'utiliser une méthode plus sophistiquée que le Naïve Bayes, appelée **Analyse Discriminante Linéaire (LDA)**. Nous rappelons que la LDA repose sur la modélisation suivante de la vraisemblance de chaque point de donnée :

$$p(x_i|y_i = k) = \frac{1}{\sqrt{(2\pi)^p|\Sigma_k|}} \exp\left(-\frac{1}{2}(x_i - \mu_k)^\top \Sigma_k^{-1}(x_i - \mu_k)\right)$$

Pour LDA, la matrice de covariance Σ est **commune à toutes les classes**. Cette hypothèse nous permet d'améliorer la séparation entre les groupes en réduisant la variance des paramètres. La distribution a posteriori devient :

$$\mathbb{P}(y_1, \ldots, y_N|x_1, \ldots, x_N) \propto \prod_{i=1}^{N} \frac{1}{\sqrt{|\Sigma|}} \exp\left(-0.5(x_i - \mu_{y_i})^T \Sigma^{-1}(x_i - \mu_{y_i})\right) \cdot \pi_{y_i}.$$

L'estimateur $\hat{\mu}_k$ est obtenu en maximisant la vraisemblance :

$$\hat{\mu}_k = \frac{1}{N_k} \sum_{i:y_i=k} x_i.$$

Intuition : La moyenne empirique minimise l'erreur quadratique, ce qui correspond au maximum de vraisemblance pour une Gaussienne.

L'estimateur de la probabilité a priori de la classe k est ensuite trouvé en utilisant la méthode des multiplicateurs de Lagrange après avoir maximisé le logarithme de vraissemblance :

$$logP(y_1, ..., y_N) = \sum_{k=0}^{K-1} N_k log\pi_k$$

Ainsi avec la contrainte :

$$\sum_{k=0}^{K-1} \pi_k = 1$$

Nous avons finalement :

$$\hat{\pi}_k = \frac{N_k}{N}.$$

Il représente la proportion d'échantillons appartenant à la classe k dans l'ensemble d'apprentissage. Cet estimateur nous est utile dans la décision finale pour ajuster la probabilité a posteriori d'une classe en fonction de sa fréquence d'apparition. Pour $\hat{\Sigma}_k$, on calcule la covariance empirique pondérée par classe :

$$\hat{\Sigma}_k = \frac{1}{N_k} \sum_{i:y_i=k} (x_i - \hat{\mu}_k)(x_i - \hat{\mu}_k)^T.$$

Nous voulons montrer que la règle de décision pour LDA s'obtient en maximisant le logarithme de la vraisemblance :

$$y^* = \arg\max_k \left[-0.5(x^* - \hat{\mu}_k)^T \Sigma^{-1} (x^* - \hat{\mu}_k) + \log \pi_k\right].$$

Nous commençons en utilisant la loi de Bayes pour exprimer la probabilité a posteriori d'une classe :

$$P(y = k|x^*) \propto P(x^*|y = k)\pi_k.$$

En remplaçant la vraisemblance $P(x^*|y = k)$ par la densité gaussienne :

$$P(x^*|y = k) = \frac{1}{\sqrt{|\Sigma|}} \exp\left(-0.5(x^* - \mu_k)^T \Sigma^{-1} (x^* - \mu_k)\right),$$

et en prenant le logarithme, on obtient :

$$\log P(y = k|x^*) = -0.5(x^* - \mu_k)^T \Sigma^{-1} (x^* - \mu_k) + \log \pi_k + C$$

où C est une constante indépendante de k. Maximiser cette expression revient à maximiser uniquement :

$$-0.5(x^* - \mu_k)^T \Sigma^{-1} (x^* - \mu_k) + \log \pi_k.$$

Ainsi, la règle de décision finale s'écrit :

$$y^* = \arg\max_k \left[-0.5(x^* - \hat{\mu}_k)^T \Sigma^{-1} (x^* - \hat{\mu}_k) + \log \pi_k\right].$$

Contrairement à Naïve Bayes, LDA utilise une matrice de covariance complète, capturant les corrélations entre caractéristiques.

Listing 4: Implémentation de LDA

```python
class LDA:
    def __init__(self, param = 1e-6):
        self.param = param

    def fit(self, X, y):
        self.classes = np.unique(y)
        n_features = X.shape[1]
        self.means = np.zeros((len(self.classes), n_features))
        self.priors = np.zeros(len(self.classes))
        self.cov = np.zeros((n_features, n_features))

        for idx, c in enumerate(self.classes):
            X_c = X[y == c]   # comme pour Bayes, on garde ce qui est de classe c
            # les 3 estimateurs :
            self.means[idx] = np.mean(X_c, axis=0)  # mu_k
            self.priors[idx] = X_c.shape[0] / X.shape[0]  # pi_k
        self.cov = np.cov(X.T) + np.eye(n_features) * self.param  # Sigma_k * param  (pour etre sur de
            pouvoir l'inverser dans _predict)

```

```python
        # on deplace l'inversion ici pour avoir la progression de _predict correcte et bien voir la fin du calcul d'inverse
        print("Inversion de la matrice de covariance en cours...")
        self.inv_cov = np.linalg.inv(self.cov) # on inverse Sigma_k
        print("Inversion terminee")

    def predict(self, X):
        y_pred = []
        for x in tqdm(X, desc="Prediction LDA en cours"):
            y_pred.append(self._predict(x))
        return np.array(y_pred)

    def _predict(self, x):
        posteriors = []

        for idx, c in enumerate(self.classes): # pareil que NaiveBayes, on separe y* ici
            quad = -0.5 * (x - self.means[idx]).T @ self.inv_cov @ (x - self.means[idx])
            prior = np.log(self.priors[idx])
            posteriors.append(quad + prior)
        return self.classes[np.argmax(posteriors)]
```

1.2.4 Pour aller plus loin !

Modèle	Précision (0)	Précision (1)	Rappel (0)	Rappel (1)
KNN	0.92	1.00	1.00	0.44
Naïve Bayes	0.99	0.84	0.97	0.92
LDA	0.96	1.00	1.00	0.72

Nous rappelons les définitions des métriques utilisées :
- **Précision** : proportion de vrais positifs parmi les prédictions positives.

$$\text{Prec} = \frac{\text{VraiPositif}}{\text{VraiPositif} + \text{FauxPositif}}$$

- **Rappel** : proportion de vrais positifs détectés parmi tous les exemples de cette classe.

$$\text{Rec} = \frac{\text{VraiPositif}}{\text{VraiPositif} + \text{FauxNégatif}}$$

Le F1-score est une moyenne harmonique entre la précision et le rappel :

$$\text{F1-score} = \frac{2 \times \text{Prec} \times \text{Rec}}{\text{Prec} + \text{Rec}}$$

Modèle	F1-score (0)	F1-score (1)	Macro avg	Weighted avg
KNN	0.96	0.61	0.79	0.91
Naïve Bayes	0.98	0.88	0.93	0.97
LDA	0.98	0.84	0.91	0.96

- **KNN** est inefficace pour détecter les spams. Sa précision est élevée, ce qui signifie qu'il ne se trompe pas lorsqu'il identifie un spam, mais son rappel faible (0.44) montre qu'il en laisse passer beaucoup. Son F1-score sur les spams est donc très bas.
- **Naïve Bayes** offre un bon compromis. Son rappel est bien plus élevé (0.92), bien qu'il commette quelques erreurs en classant des ham en spam. Cependant, il ne rate presque aucun spam, avec un taux de faux négatifs de seulement 8%.
- **LDA** est plus strict et ne fait aucune erreur sur les spams qu'il détecte (précision de 1.00), mais il en manque encore 28% (rappel de 0.72). Cela réduit son F1-score pour les spams par rapport à Naïve Bayes.

On pourrait donc se demander pourquoi la LDA ne surpasse pas Naïve-Bayes ?
Au vu de la progression, nous nous attendions à ce que LDA surpasse Naïve Bayes.
Cependant, ce n'est pas le cas. Nous avons essayé de rechercher des raisons pouvant expliquer cela, en supposant que cela ne vienne pas d'une erreur dans notre code :

- **Problème d'encodage des données** : L'utilisation de `CountVectorizer()` encode des fréquences brutes ce qui pourrait ne pas forcément être le plus adapté à LDA.
- **Nature des modèles** : Naïve Bayes serait souvent plus adapté au traitement des données textuelles que LDA.
- **Hypothèses des modèles** : Naïve Bayes pourraient simplement reposer sur des hypothèses plus adaptées ici, alors que celles de LDA étant plus strictes seraient moins efficaces dans ce cas précis.

Pour continuer, nous avons tracé les nuages de points des différentes méthodes :

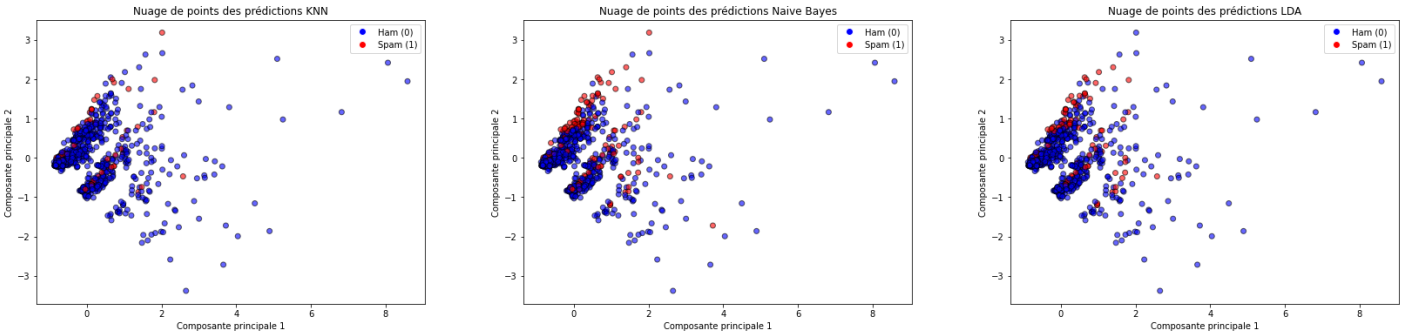

Figure 1: Nuage de points des différentes méthodes.

Les résultats obtenus confirment les tendances observées dans les tableaux précédents. Le modèle KNN présente peu de points rouges comparé aux deux autres, ce qui est cohérent avec sa précision de 100 % sur les spams mais son rappel à 64%. Il manque 56 % des spams, d'où ce manque de spam dans le nuage.
Naïve Bayes et LDA affichent des comportements similaires, bien que Naïve Bayes présente un nombre légèrement plus élevé de points rouges. Cette différence se retrouve dans les statistiques : Naïve Bayes a une précision inférieure à celle de LDA, mais un rappel nettement plus élevé. En effet, il ne manque que 8 % des spams contre 28 % pour LDA, malgré la précision parfaite de ce dernier.
Ces observations mettent en évidence les compromis entre précision et rappel selon les modèles, illustrant leurs forces et faiblesses respectives dans la détection des spams.

2 Détection de cancer du sein

Dans cette section, nous proposons de travailler avec un nouveau jeu de données. Il contient des informations médicales sur plusieurs patients potentiellement atteints d'un cancer du sein. Ces informations sont des caractéristiques (30) calculées à partir d'une image numérisée d'une aspiration à l'aiguille fine (FNA) d'une masse mammaire et décrivent les caractéristiques des noyaux cellulaires présents dans l'image.

Sur la base de ces caractéristiques, un diagnostic a été établi pour déterminer si la tumeur est maligne ou bénigne. Si la tumeur est maligne, les caractéristiques sont associées à la classe 1 ; si elle est bénigne, elles sont associées à la classe 0. Ensuite, nous cherchons à utiliser ces données afin de prédire automatiquement si un patient est atteint de cette pathologie.

2.1 Pre-Processing

Ouverture du fichier
Nous ouvrons tout d'abord le fichier Python nommé `PartII.py`, qui contient le code à compléter pour la suite de l'analyse.
Chargement des données
Nous chargeons les données en utilisant la commande `load breast cancer`. Cela permet de récupérer les caractéristiques $\{x_i\}_{i=1}^N$ ainsi que la classe associée $\{y_i\}_{i=1}^N \in \{0,1\}$. Ces données représentent des mesures issues d'images numérisées de cellules tumorales et permettent de déterminer si une tumeur est bénigne (0) ou maligne (1).
Normalisation des caractéristiques
Nous procédons à la normalisation des caractéristiques à l'aide de la fonction `StandardScaler`. Cette étape permet d'harmoniser les échelles des différentes variables, ce qui améliore la performance des algorithmes d'apprentissage automatique.

Listing 5: Chargement du jeu de données Breast Cancer

```python
#---------------------------------PROJET: PART II-----------------------#

import numpy as np
import pandas as pd
from sklearn.preprocessing import StandardScaler
from sklearn.model_selection import train_test_split
from sklearn.metrics import accuracy_score, classification_report
from sklearn.datasets import load_breast_cancer

from PartI_modif import LDA #pour eviter de copier coller la class ici


data = load_breast_cancer() #Charge les donnees sous le nom de "data"
X = data.data # X represente les entrees/features de notre modele
y = data.target # y represente les valeurs cibles que nous voulons predire

# STANDARDIZE THE DATA
scale = StandardScaler() #standartScaler transforme nos donnees en faisant X-moyenne/ecarttype (centre
    les donnees a 0 et met la variance a 1)
X = scale.fit_transform(X)  #comme dans la PI on normalise nos donnees

X_train, X_test, y_train, y_test = train_test_split(X, y, test_size=0.2, random_state=42) # comme pour
    la PI on separe a 0.8/0.2 entre train et test
```

2.2 Un peu de théorie sur la régression logistique

Nous proposons dans cette section d'implémenter une nouvelle méthode de classification appelée régression logistique. L'idée de cette approche est toujours de supposer un modèle statistique mais directement paramétré sur la distribution a posteriori

$$P(y_i = 1|x_i, \theta) = \frac{1}{1 + \exp\left(-\left(\theta^0 + \sum_{p=1}^{P} \theta_p x_i^p\right)\right)}$$

où $\theta = \{\theta^0, \theta_1, \ldots, \theta_P\} \in \mathbb{R}^P$ est l'ensemble des paramètres inconnus à apprendre.

$$\mathbb{P}(y_i = 0|x_i, \boldsymbol{\theta}) = 1 - \mathbb{P}(y_i = 1|x_i, \boldsymbol{\theta}) = \frac{1}{1 + \exp\left(-\left(\theta^0 + \sum_{p=1}^{P} \theta^p x_i^p\right)\right)}$$

Par définition de la régression logistique :

$$P(y_i = 1|x_i, \theta) = \frac{1}{1 + \exp\left(-\left(\theta^0 + \sum_{p=1}^{P} \theta^p x_i^p\right)\right)}$$

et donc :

$$P(y_i = 0|x_i, \theta) = 1 - P(y_i = 1|x_i, \theta)$$

En supposant les données indépendantes conditionnellement à θ :

$$P(y_1, \ldots, y_N|x_1, \ldots, x_N, \theta) = \prod_{i=1}^{N} P(y_i|x_i, \theta)$$

Pour chaque $y_i \in \{0, 1\}$, on peut écrire :

$$P(y_i|x_i, \theta) = \begin{cases} P(y_i = 1|x_i, \theta) & \text{si } y_i = 1 \\ P(y_i = 0|x_i, \theta) & \text{si } y_i = 0 \end{cases}$$

Ce qui se condense en :

$$P(y_i|x_i, \theta) = P(y_i = 1|x_i, \theta)^{y_i} \cdot P(y_i = 0|x_i, \theta)^{1-y_i}$$

En remplaçant $P(y_i = 0|x_i, \theta)$:

$$P(y_1, \ldots, y_N|x_1, \ldots, x_N, \theta) = \prod_{i=1}^{N} \left[P(y_i = 1|x_i, \theta)^{y_i} \cdot (1 - P(y_i = 1|x_i, \theta))^{1-y_i}\right]$$

Ce résultat correspond à une loi binomiale. Ainsi :

$$\boxed{P(y_1,\ldots,y_N|x_1,\ldots,x_N,\theta)=\prod_{i=1}^{N}P(y_i=1|x_i,\theta)^{y_i}(1-P(y_i=1|x_i,\theta))^{(1-y_i)}}$$

L'intérêt de la régression logistique est de **modéliser directement la frontière de décision** entre classes via une fonction sigmoïde (une fonction permettant de garder la sortie entre 0 et 1), évitant les hypothèses fortes sur la distribution des données (contrairement à LDA ou Naïve Bayes). Elle permet aussi une interprétation des coefficients $\boldsymbol{\theta}$. Pour effectuer une prédiction, nous devons d'abord apprendre/estimer l'ensemble des paramètres θ sur l'ensemble d'entraînement $(x_i,y_i)_{i=1}^N$.

On pourrait estimer $\boldsymbol{\theta}$ par **maximum de vraisemblance**, mais la solution analytique est impossible car l'équation $\nabla J(\boldsymbol{\theta})=0$ est non linéaire. D'où l'utilisation d'une méthode numérique (ex : descente de gradient). Une solution courante pour déterminer θ est d'utiliser une approche numérique et de trouver une séquence de valeurs $\{\theta^{(l)}\}_{l=1}^L$ (L : nombre d'itérations) convergeant vers θ.

$$\theta^{(l+1)}=\theta^{(l)}-\alpha\nabla J(\theta^{(l)})$$

où

$$J(\theta^{(l)})=-\log P(y_1,\ldots,y_N|x_1,\ldots,x_N,\theta^{(l)})$$

et $\alpha>0$ est le taux d'apprentissage.
Calculer le gradient de la fonction coût $J(\boldsymbol{\theta})$ pour la régression logistique :

$$\frac{\partial J(\boldsymbol{\theta})}{\partial\theta^p}=\sum_{i=1}^{N}\left(\mathbb{P}(y_i=1|\mathbf{x}_i,\boldsymbol{\theta})-y_i\right)x_i^p\quad\forall p\in\{1,\ldots,P\}$$

$$\frac{\partial J(\boldsymbol{\theta})}{\partial\theta^0}=\sum_{i=1}^{N}\left(\mathbb{P}(y_i=1|\mathbf{x}_i,\boldsymbol{\theta})-y_i\right)$$

D'après les équations, la vraisemblance s'écrit :

$$J(\boldsymbol{\theta})=-\log\mathcal{L}(\boldsymbol{\theta})=-\sum_{i=1}^{N}\left[y_i\log\sigma(\boldsymbol{\theta}^\top\tilde{\mathbf{x}}_i)+(1-y_i)\log(1-\sigma(\boldsymbol{\theta}^\top\tilde{\mathbf{x}}_i))\right]$$

où $\tilde{\mathbf{x}}_i=[1,x_i^1,\ldots,x_i^P]^\top$ et $\sigma(z)=\frac{1}{1+e^{-z}}$.
Pour $p\in\{0,1,\ldots,P\}$, on dérive terme à terme :

$$\begin{aligned}\frac{\partial J}{\partial\theta^p}&=-\sum_{i=1}^{N}\left[\frac{y_i}{\sigma(\boldsymbol{\theta}^\top\tilde{\mathbf{x}}_i)}\frac{\partial\sigma}{\partial\theta^p}-\frac{1-y_i}{1-\sigma(\boldsymbol{\theta}^\top\tilde{\mathbf{x}}_i)}\frac{\partial\sigma}{\partial\theta^p}\right]\\&=-\sum_{i=1}^{N}\frac{\partial\sigma}{\partial\theta^p}\left[\frac{y_i}{\sigma_i}-\frac{1-y_i}{1-\sigma_i}\right]\end{aligned}$$

En utilisant $\frac{\partial\sigma(z)}{\partial z}=\sigma(z)(1-\sigma(z))$:

$$\frac{\partial\sigma(\boldsymbol{\theta}^\top\tilde{\mathbf{x}}_i)}{\partial\theta^p}=\sigma_i(1-\sigma_i)\tilde{x}_i^p$$

En substituant dans l'équation précédente :

$$\begin{aligned}\frac{\partial J}{\partial\theta^p}&=-\sum_{i=1}^{N}\sigma_i(1-\sigma_i)\tilde{x}_i^p\left[\frac{y_i}{\sigma_i}-\frac{1-y_i}{1-\sigma_i}\right]\\&=-\sum_{i=1}^{N}\tilde{x}_i^p\left[y_i(1-\sigma_i)-(1-y_i)\sigma_i\right]\\&=\sum_{i=1}^{N}\tilde{x}_i^p(\sigma_i-y_i)\end{aligned}$$

Pour θ^0 $(p=0)$, on a $\tilde{x}_i^0=1$:

$$\frac{\partial J}{\partial\theta^0}=\sum_{i=1}^{N}(\sigma_i-y_i)$$

donc :

$$\boxed{\begin{aligned} \frac{\partial J}{\partial \theta^p} &= \sum_{i=1}^{N} (\mathbb{P}(y_i = 1|\mathbf{x}_i, \boldsymbol{\theta}) - y_i) x_i^p \quad \forall p \geq 1 \\ \frac{\partial J}{\partial \theta^0} &= \sum_{i=1}^{N} (\mathbb{P}(y_i = 1|\mathbf{x}_i, \boldsymbol{\theta}) - y_i) \end{aligned}}$$

Une fois les paramètres estimés, la prédiction de classe peut être effectuée. Cela implique naturellement de calculer :

$$P(y_i = 1|x_\star, \hat{\theta})$$

et d'affecter $x^\star$ à la classe 1 si $P(y_i = 1|x^\star, \hat{\theta}) > 0.5$.

2.3 implémentation RL

Dans cette section, nous proposons de mettre en œuvre la méthode sur notre jeu de données sur le cancer du sein. Comme précédemment, une classe `RegressionLogistic` a été pré-créée. Le constructeur de cette classe inclut deux attributs : le nombre d'itérations(`num_iterations`) et le taux d'apprentissage (`learning_rate`), qui seront utilisés par la suite.

Listing 6: Implementation de la regression logistique personnalisée

```python
class LogisticRegressionCustom:
    def __init__(self, learning_rate, num_iterations):
        self.learning_rate = learning_rate
        self.num_iterations = num_iterations

    def sigmoid(self, z):
        return 1/(1 + np.exp(-z))

    def fit(self, X, y):
        self.m, self.n = X.shape
        self.weights = np.zeros(self.n) # cree un vecteur 0
        self.bias = 0
        self.X = X
        self.y = y

        ## la boucle tournera autant de fois qu'il y a de nombres d'iterations, elle execute update_weights() a chaque fois
        for i in range(self.num_iterations):
            self.update_weights()

    def update_weights(self):
        # TO DO
        self.modele_lin = self.X@self.weights+self.bias
        y_pred = self.sigmoid(self.modele_lin) # on applique le calcul defini dans sigmoid sur modele_lin
        # on definit les 2 gradients :
        dw = (1/self.m)*self.X.T@(y_pred- self.y) # poids
        db = (1/self.m)*np.sum(y_pred-self.y) # biais (np.sum fait la somme des valeurs d'un tableau sur un axe)
        # on applique une iteration
        self.weights -= self.learning_rate*dw # - parce qu'on reduit a chaque maj pour arriver a la meilleure coupe entre les classes
        self.bias -= self.learning_rate*db

    def predict(self,X):
        modele_lin = X@self.weights+self.bias # on le redef comme dans fit mais pour qu'il soit bien fait avec X_test ici
        y_pred = self.sigmoid(modele_lin) # y_pred mais avec modele_lin calcule avec X_test
        Y = (y_pred >= 0.5).astype(int) # seuil pour classer defini ici a 0.5 (dans le sujet)
        return np.array(Y)


# %% Question 11 avec accuracy score
print("Question 11 : Logistic Regression")
iterations = [1000,5000,10000,15000,20000,]
# on fait une boucle pour tester les differents nombres d'iterations
for i in iterations :
    log_reg = LogisticRegressionCustom(learning_rate = 0.01, num_iterations = i)
    log_reg.fit(X_train, y_train)
    y_pred = log_reg.predict(X_test)
    print(f'\nNombres d\'iterations : {i}') # et on affiche les resultats
    print(f'Accuracy : {accuracy_score(y_test, y_pred):.4f}')

# %% Question 12 avec accuracy score

lda = LDA()
lda.fit(X_train, y_train)
y_pred_lda = lda.predict(X_test)

print("\nQuestion 12 : comparaison avec LDA")
print(f'Accuracy : {accuracy_score(y_test, y_pred_lda):.4f}')

# %% graphe comparaison it
import matplotlib.pyplot as plt

iterations = [10,20,250,270,300,800,1000,1200,1300,1500,1800,2000]
acc = []

for i in iterations:
    log_reg = LogisticRegressionCustom(learning_rate=0.01, num_iterations=i)
    log_reg.fit(X_train, y_train)
    y_pred = log_reg.predict(X_test)
    acc.append(accuracy_score(y_test, y_pred))

plt.plot(iterations, acc, marker='o')
plt.xlabel("Nombre d'iterations")
```

```python
plt.ylabel("Accuracy sur X_test")
plt.title("Impact des iterations sur l'accuracy")
plt.grid()
plt.show()

# permet de visualiser la "chute" d'accuracy (~1%)
# %% Question 11 avec classification_report
print("Question 11 : Logistic Regression")
iterations = [1000,5000,10000,20000]
for i in iterations :
    log_reg = LogisticRegressionCustom(learning_rate = 0.01, num_iterations = i)
    log_reg.fit(X_train, y_train)
    y_pred = log_reg.predict(X_test)
    print(f'Nombres d\'iterations : {i}')
    print(classification_report(y_test, y_pred))

# %% Question 12 avec classification_report

lda = LDA()
lda.fit(X_train, y_train)
y_pred_lda = lda.predict(X_test)

print("\nQuestion 12 : comparaison avec LDA")
print(classification_report(y_test, y_pred_lda))
```

Le graphe ci-dessous montre l'évolution de l'accuracy en fonction du nombre d'itérations pour la méthode de Régression Logistique.

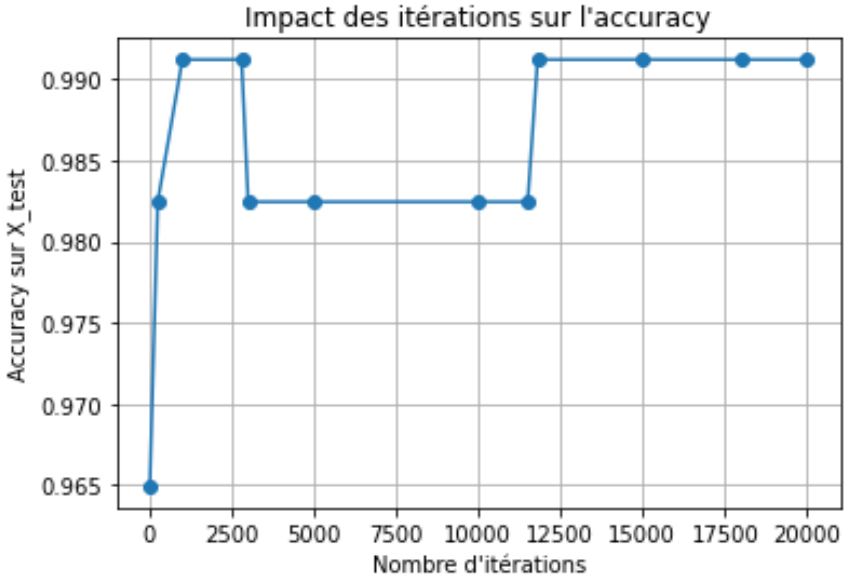

Figure 2: Impact des itérations sur l'accuracy pour la Régression Logistique.

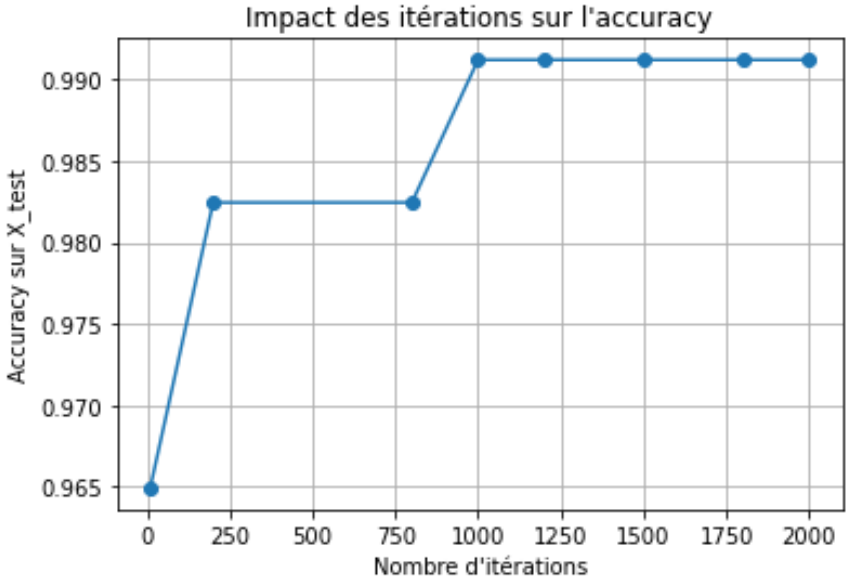

Figure 3: Progression de l'accuracy entre 10 et 2000 itérations.

- **Convergence rapide** : L'accuracy atteint rapidement un niveau élevé (˜0.99) en 1000 itérations.
- **Phase de stagnation** : Après cette montée initiale, la performance reste stable, ce qui indique une bonne convergence.
- **Légère baisse** : Une diminution temporaire de l'accuracy de moins de 1% est observée entre 2500 et 11 500 itérations.
- **Stabilité finale** : Après 12 000 itérations, l'accuracy reste stable et ne subit plus de variations significatives.

Conclusion

Ce projet a permis de voir et de comparer différentes méthodes d'implémentation de l'IA grâce à Python. Ici appliquées à deux problématiques distinctes : la détection de spams et le diagnostic du cancer du sein. Les algorithmes testés (KNN, Naïve Bayes, LDA et régression logistique) ont été évalués, révélant des performances pouvant varier selon les contextes mais aussi les attentes.

Synthèse des résultats

- **KNN** a montré des performances bien moins élevées que les autres, en particulier avec un rappel très faible pour les spam.
- **Naïve Bayes et LDA**, malgré des hypothèses simplificatrices (indépendance des variables pour Naïve Bayes, normalité des données pour LDA), ils ont obtenu des résultats solides avec des temps de calcul réduits
- **La régression logistique** s'est démarquée de par sa stabilité et sa capacité à bien séparer les classes, confirmant son efficacité dans des problèmes de classification binaire.

Ce que nous avons appris

Ce travail a permis de s'imprégner du fonctionnement de différents modèles de classification ainsi que de comprendre un peu mieux comment s'en servir efficacement de par les forces et faiblesses qu'ils peuvent avoir. Il a également permis de nous souligner l'importance d'avoir une démarche critique dans l'interprétation des résultats. Nous avons pu voir qu'un bon score global ne nous garantit pas une bonne performance sur toutes les classes. Cela peut être déterminant dans des contexte vitaux, comme un diagnostic médical (faux négatif) mais aussi pour des taches plus basiques comme la détection de spam (censure de mails en tant que spam).

Pour aller plus loin

Pour prolonger cette étude, nous avons quelques pistes que nous pourrions explorer :

Approches plus poussées

- Implémentation de modèles d'ensemble (Random Forest, Gradient Boosting) ou de réseaux de neurones.
- Améliorer la représentation des textes avec des techniques de traitement du langage naturel (TF-IDF, Word2Vec) pour mieux détecter les spams.

Explicabilité

- Intégration d'outils comme SHAP[1] ou LIME[2] afin de mieux comprendre les décisions des modèles, notamment dans un cadre médical où la transparence est cruciale.

Ce projet montre bien que si les modèles classiques offrent une base solide, les défis auquels nous nous retrouvons dans le monde réel demandent souvent des méthodes plus poussées et sophistiquées mais aussi une réflexion critique sur l'interprétabilité et l'éthique de l'IA.

[1] SHAP : (SHapley Additive exPlanations) est une méthode expliquant les prédictions d'IA en calculant l'impact de chaque variable (ex: "le mot 'gratuit' augmente de 30% la probabilité que l'email soit un spam").

[2] LIME : Approche simplifiée pour comprendre une prédiction précise en créant un modèle local autour d'un cas (ex: "ce diagnostic de cancer dépend surtout de la taille de la tumeur").